

# Incremental Maintenance of RDB2RDF Views: A Post-Update, LLM-Compilable Formulation of Changeset Computation

1st. Author  
e-mail  
Intitution  
City, State, Country

2nd. Author  
e-mail  
Intitution  
City, State, Country

## Abstract

A strategy for exposing relational data as part of a knowledge graph is to define RDB2RDF views that map relational tuples to RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability, requiring incremental mechanisms to maintain them under updates. A central problem in incremental maintenance is computing deletion changesets, which traditionally requires reconstructing the pre-update database state. This paper challenges this assumption by introducing a formal model of transformation rules that leads to provably correct changesets that operate directly over the post-update state. The paper proceeds by describing how to create an asynchronous maintenance tool for an RDB2RDF view, defined by R2RML statements, leveraging the formal model. As a pre-processing stage, the paper starts by showing how to compile R2RML statements into transformation rules with the help of an LLM, prompted with the formal model. Then, it introduces an asynchronous maintenance architecture that features AFTER triggers automatically synthesized from the transformation rules, again with the help of an LLM, prompted with the formal model. Finally, the paper describes an experiment that evaluates the complete implementation cycle for an RDB2RDF view defined on top of a version of the MusicBrainz database. The paper therefore demonstrates that LLMs, when guided by the proposed formal model, can automatically generate correct database triggers and the maintenance logic for the proposed architecture.

## CCS Concepts

• Information systems → Data replication tools; • Computing methodologies → Language resources.

## Keywords

LLM SQL Code Generation, Semantic View, Enterprise Knowledge Graph, Formal Specification

## ACM Reference Format:

1st. Author and 2nd. Author. 2018. Incremental Maintenance of RDB2RDF Views: A Post-Update, LLM-Compilable Formulation of Changeset Computation. In *Proceedings of EDBT/ICDT 2027 Joint Conference (EDBT/ICDT 2027)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 Introduction

Enterprise Knowledge Graphs (EKGs) have emerged as a powerful paradigm for integrating heterogeneous enterprise data sources into a unified semantic layer. By linking structured and semi-structured data via an ontology, EKGs enable applications to query and analyze enterprise information in a semantically rich, integrated manner. In many real-world deployments, the underlying enterprise data remains stored in relational databases, while the knowledge graph provides a semantic access layer on top of these sources.

A common strategy for exposing relational data as part of a knowledge graph is the definition of *RDB2RDF views*, which map relational tuples into RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability. However, when the relational source is frequently updated, the materialized RDB2RDF view must be continuously synchronized with the underlying database. Recomputing the entire RDB2RDF view after each update is typically impractical, motivating the need for *incremental maintenance techniques* that compute only the changes to the view.

A well-established approach for incremental view maintenance is to compute *changesets*, represented as a pair  $\langle \Delta^-(u), \Delta^+(u) \rangle$ , capturing the triples removed from and added to the RDB2RDF view after a relational update  $u$ . Prior work [4] has shown that, for entity-preserving RDB2RDF statements, it is sufficient to identify the *pivot tuples* whose RDF contributions may be affected by the update and recompute their associated RDF state.

The formal framework introduced in [4] provides a rigorous method for computing such changesets based on transformation rules over relational schemas. In particular, in this framework, the deletion component  $\Delta^-(u)$  is defined with respect to the pre-update database state. Conceptually, this definition is correct, as it captures precisely the RDF triples that must be removed from the view. However, it introduces a practical limitation when implementing the framework using database triggers, namely, relying on the pre-update database state.

This paper challenges this assumption by introducing a formal model that leads to provably correct changesets that operate directly over the post-update state. It shows that, for a transformation rule without multiple occurrences of the same relation, both deletion and insertion components can be derived using only the post-update state, the transition tables, and the pre-update view state. For a

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted by ACM, provided that the copies are not made for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

EDBT/ICDT 2027, Lille, France

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

2026-07-25 20:32. Page 1 of 1–13.

transformation rule with multiple occurrences of the same relation  $R$ , it captures the impact of deletions and insertions on  $R$  directly in the evaluation of the transformation rule, still without using the pre-update database state.

The paper proceeds by describing how to create an asynchronous maintenance tool for an RDB2RDF view, defined by R2RML statements, leveraging the formal model. As a pre-processing stage, the paper starts by showing how to compile R2RML statements into transformation rules with the help of an LLM, prompted with the formal model. Then, it introduces an asynchronous maintenance architecture that selectively decouples SPARQL computation from relational transactions. The architecture avoids executing SPARQL queries inside relational transactions, preventing triplestore latency or temporary unavailability from blocking database updates, while preserving correctness and improving scalability. Finally, it discusses how to compile the required AFTER triggers from the transformation rules, again with the help of an LLM, prompted with the formal model. In combination, this means that, given an RDB2RDF view defined by R2RML statements, the underlying relational database can be equipped with AFTER triggers that, combined with the other (fixed) components of the architecture, incrementally maintain the RDB2RDF view.

Finally, the paper describes an experiment that evaluates the complete implementation cycle for an RDB2RDF view defined on top of a version of the MusicBrainz database. The database features a schema with 374 relations that encompass information about artists, release groups, etc. The RDB2RDF view uses the *Music Ontology* vocabulary and is defined by 57 R2RML statements.

The paper therefore demonstrates that LLMs, when guided by the proposed formal model, can act as compilers, automatically generating correct database triggers and the maintenance logic for the proposed architecture. The experiments showcase that LLMs are already capable of processing a (complex and lengthy) formal model and correctly output the specified database code, supporting a new paradigm of specification-driven, LLM-based synthesis for incremental RDB2RDF view maintenance.

To summarize, the first contribution of this paper is a formal model of the deletion and insertion components of changesets directly over the post-update database state. The second contribution is to describe how to create asynchronous maintenance tools for RDB2RDF views, defined by R2RML statements. The third contribution is to demonstrate that an LLM, guided by appropriate prompts and accessing the formal model, can generate database code that computes and publishes provably correct changesets.

The paper is organized as follows. Section 2 covers related work. Section 3 introduces the notion of entity-preserving RDB2RDF views. Section 4 summarizes the theory of incremental RDB2RDF view maintenance. Section 5 discusses implementation details. Section 6 describes the experiments conducted and discusses the results. Finally, Section 7 contains the conclusions.

## 2 Related Work

**Incremental View Maintenance (IVM).** The Incremental View Maintenance problem has been studied in the literature for a very long time [15].

Early work covered relational views [9], object-oriented views [2], semi-structured views [24], and XML views [12], just to name a few efforts.

Some more recent approaches include the following. As for generic IVM tools, DBSP [8] introduces an expressive language that enables efficient incremental view maintenance for queries written in several languages, such as SQL and Datalog. Enzyme [23] is an IVM engine, developed at Databricks to power Spark Declarative Pipelines, that provides a built-in, end-to-end approach to incremental pipelines, utilizing materialized views as first-class building blocks. OpenIVM [7] is an SQL-to-SQL compiler for Incremental View Maintenance (IVM) that compiles view definitions into SQL to propagate deltas into the table that materializes the view eventually.

As for IVM SPARQL and graph databases, Incremunica [21] integrates multiple incremental operators, allowing it to adapt to different queries and data for optimal performance, and supporting federated querying, dynamically adding and removing sources during query execution, SPARQL Query Language support, multiple IVM techniques, and client-side execution. The counting algorithm, a classic approach to incremental view maintenance for queries on relational data, is adapted in [19] to SPARQL queries on RDF datasets. Graph property views, both virtual and materialized, are considered in [10] for integrating heterogeneous data using rewriting techniques.

Solutions in specific settings include the following. The maintenance of self-join-free conjunctive queries over a dynamic database, where tuples can be inserted or deleted, is investigated in [11]. A maintenance approach that combines delta queries, trees of materialized views, and heavy-light data partitioning is introduced in [1], whose update time matches or improves the best update time reported in prior work. The incorporation of knowledge about foreign keys into IVM to speed up performance is investigated in [20], which reports experiments in Redshift showing that the proposed technique improved the execution times of the whole refresh process up to 2 times, and up to 2.7 times the process of calculating the necessary changes that will be applied to the materialized view. Finally, a framework for maintaining sketches incrementally under updates is introduced in [13].

The main idea that distinguishes the formal model proposed in this paper from previous work on incremental view maintenance is to explore the entity-preservation property of typical RDB2RDF views, which enables the identification of the specific tuples relevant to a data source update. Only the portion of the view associated with the relevant tuples should be re-materialized.

The proposed formal model revises earlier work [5], which described an incremental maintenance strategy, based on triggers, for RDB2RDF views defined on top of relational data. This earlier work was further modified and enhanced to compute changesets for RDB2RDF views [4]. However, these earlier approaches depended on accessing the database state before the update to generate the view changesets. The formal model proposed in this paper remedies this situation and computes the required view changesets from the post-update database state, the current view state, and the update itself.

The definition of the formal model co-evolved with the prompts used in the experiments in the following sense. The experiment

started with simple prompts, with the original version of the formal model as an attachment. An analysis of the results indicated that some errors were due to overly simple prompts, on the one hand, and to missing or poorly formulated definitions, on the other. The analysis also pointed out that the formal definitions could be changed to induce simpler database code. The prompts and the formal model were altered accordingly, and the tests were repeated until satisfactory results were achieved.

**Specification-guided LLM synthesis.** Specification-guided LLM synthesis refers to the process of generating executable artifacts from formally defined semantic specifications, where the generative model is constrained to preserve the structural and logical properties of the specification.

For example, Murphy et al. [14] divided code generation into two parts: one handled by an LLM and the other by formal methods-based program synthesis. VERGE [18] decomposes LLM outputs into atomic claims, autoformalizes them into first-order logic, and verifies their logical consistency using automated theorem proving. SGCR [22] is a framework that grounds LLMs in human-authored specifications to produce trustworthy and relevant feedback.

Specifically with respect to SQL code generation, the text-to-SQL task, that is, the generation of SQL from Natural Language descriptions, has received considerable attention. Recent LLM-based text-to-SQL tools are surveyed, for example, in [16, 17]. The Text-to-SQL Handbook<sup>1</sup> lists the best-performing text-to-SQL tools on several benchmarks.

This paper goes in a different direction and shows that LLMs are capable of processing a (complex and lengthy) formal model and correctly output the formally specified database code. Specifically, the paper shows how to compile R2RML statements into transformation rules with the help of an LLM, prompted with the formal model. It proceeds to demonstrate how to compile the required AFTER triggers from the transformation rules, again with the help of an LLM, prompted with the formal model.

### 3 Formal Specification of Entity-Preserving RDB2RDF Views

This section summarizes the major concepts and results of a theory of incremental RDB2RDF view maintenance, presented in detail in the companion full paper [3]. The theory is the basis for the incremental view maintenance tool described in Section 5.

#### 3.1 Entity-Preserving RDB2RDF Views

An *RDB2RDF view* is a triple  $\mathcal{W} = (V, \mathcal{S}, M)$ , where:

- $V$  is the vocabulary of the *target ontology*;
- $\mathcal{S}$  is the *source relational schema*; and
- $M$  is a set of *mappings* between  $V$  and  $\mathcal{S}$ , defined by *transformation rules* (TRs).

The semantics of an RDB2RDF view is defined in Section 3.3. A view  $\mathcal{W} = (V, \mathcal{S}, M)$  satisfies the *entity-preserving property* iff:

- the instances of the classes in  $V$  correspond to tuples in selected relations of  $\mathcal{S}$ , which are called the *pivot relations*

of  $\mathcal{W}$ , so that distinct tuples from the same pivot relation generate distinct class instances;

- the values of datatype properties in  $V$  of these instances are given by (functions of) attribute values of the corresponding tuples, or of related tuples;
- the object properties in  $V$  correspond to relationships between tuples of the pivot relations of  $\mathcal{S}$ .

A *transformation rule* (TR) of  $\mathcal{W}$  is an expression of the form  $C(x) \leftarrow Q(x)$  or of the form  $P(x,y) \leftarrow Q(x,y)$ , where  $C$  and  $P$  are class and property names in  $V$ , and  $Q(x)$  and  $Q(x,y)$  are queries over  $\mathcal{S}$  whose target clauses contain one and two variables, respectively.

The problem of generating TRs is addressed in [5, 6] and is outside of the scope of this work. Section 3.2 contains examples of TRs for the RDB2RDF view used in the experiments.

Table 1 lists a set of TR patterns that lead to the definition of relational to RDF statements that guarantee that the RDB2RDF view satisfies the entity-preserving property by construction. Table 1 defines three types of TR patterns: *Class Transformation Rule* (CTR), *Datatype Property Transformation Rule* (DTR), and *Object Property Transformation Rule* (OTR).

Intuitively, a CTR  $\Psi$  maps tuples of a relation  $R$  into RDF instances of a class  $C$ . It establishes a semantic correspondence between a pivot tuple  $r$  and an RDF resource  $x$ , such that each pivot tuple is associated with at most one RDF instance, and distinct pivot tuples generate distinct RDF resources. Thus, the mapping preserves the identity of relational entities in the RDB2RDF view.

There are two types of DTRs. A *Local DTR* has no relational path in the body so that the datatype values are obtained directly from attribute values of the pivot tuples.

A *Path DTR* has a relational path in the body, and the datatype values are obtained from tuples reachable through the relational path from the pivot tuples.

There are also two types of OTRs. A *regular OTR*  $\Psi$  defines object properties by relating an RDF instance generated from a pivot tuple  $t$  to an RDF instance generated from a pivot tuple  $u$  reached from  $t$  through a relational path in the source schema. Hence, OTRs capture relationships between entities represented in the database.

**A reification OTR relates an RDF instance generated from a pivot tuple  $t$  to a URI obtained by reification of a literal  $l$  created from attribute values of a pivot tuple  $u$  reached from  $t$  through a relational path in the source schema.**

The transformation rules in Table 1 use several concrete predicates. Four of these predicates are quite simple to define:

- $nonNull(v)$  holds iff value  $v$  is not null.
- $RDFLiteral(u,A,R,v)$  holds iff  $v$  is the literal representation of a value  $u$  of attribute  $A$  of  $R$ .
- $RDFURI(u,o)$  holds iff  $o$  denotes a URI that is the reification of the literal  $u$  (that is,  $o$  is just the literal  $u$  treated as a URI).
- $F(r,s)$  holds iff a tuple  $r$  of  $R$  is related to tuple  $s$  of  $S$  by a foreign key of the form  $F(R:L,S:K)$ .

The more complex predicates are as follows.

A *relational path*  $\varphi$  is a list of the form  $[F_1, \dots, F_n]$ , where  $F_j$  is either the name of a foreign key, indicating that the path traverses  $F_j$  from  $R_j$  and  $R_{j+1}$ , or of the form  $(F_j)^{-1}$ , indicating that the path traverses  $F_j$  from  $R_{j+1}$  to  $R_j$ , in the reverse direction. The expression  $path(\Psi)$  denotes the relational path of  $\Psi$ . A relational

<sup>1</sup>[https://github.com/HKUSTDial/NL2SQL\\_Handbook](https://github.com/HKUSTDial/NL2SQL_Handbook)

Table 1: Transformation Rule Patterns

| TR              | Transformation Rule Pattern                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CTR             | $\psi : C(s) \leftarrow R(r), URI\_R(r, s), \delta(r)$<br>where<br>- $\psi$ is the name of the transformation rule<br>- $R$ is the name of a relation or a view in the source schema $S$<br>- $r$ is a tuple variable bound to tuples of $R$<br>- $s$ is the URI associated with the tuple bound to $r$<br>- $URI\_R(r, s)$ is a URI constructor predicate as defined in the text<br>- $\delta(r)$ is an optional selection over $R$                                                                                                                                                                                                                                                              |
| Local DTR       | $\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$<br>$RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k), T([v_1, \dots, v_k], v)$<br>where<br>- $\psi$ is the name of the transformation rule<br>- $P$ is a datatype property of $V$ with domain $D$<br>- $nonNull$ and $RDFLiteral$ are predicates as defined in the text<br>- " $R_D(r), B_D[r, s]$ " is the RHS of the CTR $\psi_D$ that matches class $D$ with pivot relation $R_D$ of $\psi_D$<br>- $A_1, \dots, A_k$ are attributes of $R_D$<br>- $T$ is an optional function that transforms values of attributes $A_1, \dots, A_k$ of $R_D$ to values of property $P$ |
| Path DTR        | $\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$<br>$RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k), T([v_1, \dots, v_k], v),$<br>$\varphi(r, g)$<br>where<br>- the terms in the first two lines are as in a Local DTR<br>- $\varphi(r, g)$ is a predicate path as defined in the text                                                                                                                                                                                                                                                                                                                                    |
| Regular OTR     | $\psi : P(s, o) \leftarrow R_D(d), B_D[d, s], R_G(g), B_G[g, o], \varphi(d, g)$<br>where<br>- the terms are as in a Path DTR and<br>- " $R_G(g), B_G[g, o]$ " is the RHS of the CTR $\psi_G$ that matches class $G$ with pivot relation $R_G$ of $\psi_G$                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Reification OTR | $\psi : P(s, o) \leftarrow R_D(d), B_D[d, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$<br>$RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k), T([v_1, \dots, v_k], v),$<br>$RDFURI(u, o)$<br>where<br>- the terms in the first two lines are as in a Local DTR<br>- $RDFURI(u, o)$ is as defined in the text                                                                                                                                                                                                                                                                                                                                                       |

path  $\varphi$  induces a *predicate path*  $\varphi(x, y)$ , which is evaluated in a database state by joining the relations in the path in the directions indicated.

The *pivot relation* of a transformation rule  $\Psi$ , denoted  $pivot(\Psi)$ , is the relation  $R$  whose tuples, called *pivot tuples*, induce the URIs of the subjects of the triples generated by  $\Psi$ .

A *URI template* for  $R$  is a string pattern  $p$  with place holders  $\langle h_1, \dots, h_n \rangle$  such that, when the place holders are appropriately instantiated, the resulting string becomes a URI.

A URI  $s$  is associated with  $r$  via  $p$  iff the predicate

$$hasURI(p, \langle e_1, \dots, e_n \rangle, s)$$

holds, where

- $\langle e_1, \dots, e_n \rangle$  is an ordered sequence of expressions, where each  $e_i$  is of the form:
  - $e_i = r.A$ , where  $r.A$  denotes an attribute of the pivot tuple.

- $e_i = [FK(r, t)/t.A]$ , where  $[FK(r, t)/t.A]$  denotes the value of attribute  $A$  obtained after following the foreign key  $FK$  from the pivot tuple  $r$  to tuple  $t$  such that the foreign-key navigation must be functional from the pivot relation to the referenced relation.

- $s$  is the result of instantiating each  $h_i$  by  $e_i$  in  $p$ .

To simplify the notation, a second predicate, called a *URI constructor*, is introduced by definition:

$$URI\_R(r, s) \equiv hasURI(p, \langle e_1, \dots, e_n \rangle, s)$$

As an example, the first line of Table 1 indicates that the URIs associated with the pivot tuples of relation *artist* are defined by

$$hasURI("http://musicbrainz.org/artist/<h>", \langle r.gid \rangle, s)$$

That is, the URI  $s$  associate with the pivot tuple  $r$  is generated by replacing  $h$  by " $r.gid$ " in the template " $http://musicbrainz.org/artist/<h>$ ".



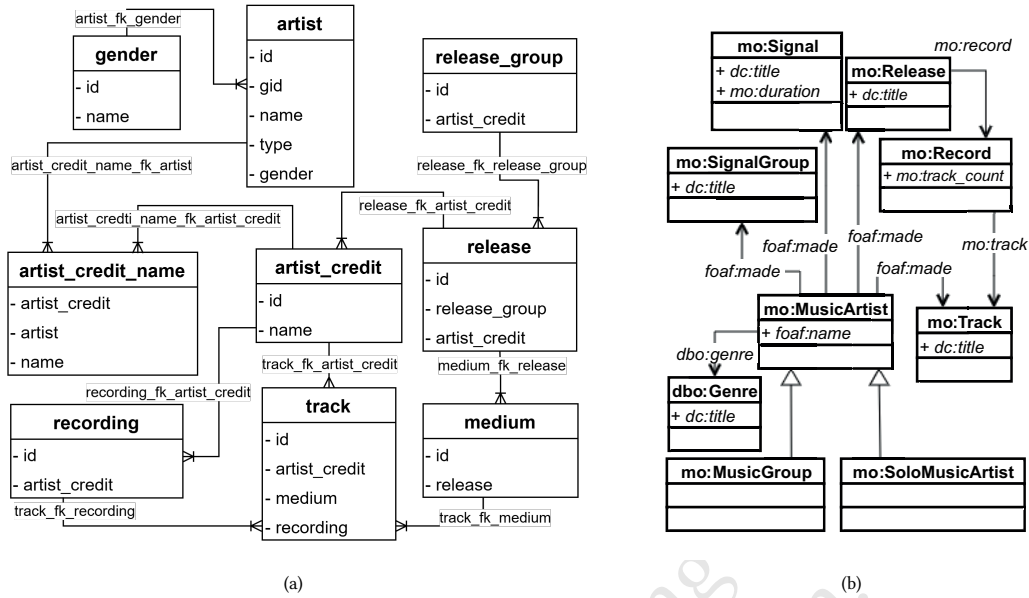
Figure 1: (a) *MusicBrainz Schema* and (b) *Fragment of LMB View Ontology*.

Table 2: Examples of the transformation rules.

| TR       | Transformation Rules                                                                                                                                                                                                    |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          | $URI\_artist(r, s) \equiv hasURI("http://musicbrainz.org/artist/\langle h \rangle", \langle r.gid \rangle, s)$                                                                                                          |
|          | $URI\_artist\_tag(r, s) \equiv hasURI("http://musicbrainz.org/tagging/\langle h_1, h_2 \rangle",$<br>$\langle [artist\_tag\_fk\_artist(r, a)/a.gid], [artist\_tag\_fk\_tag(r, t)/t.name] \rangle, s)$                   |
|          | $URI\_medium(r, s) \equiv hasURI("http://musicbrainz.org/record/\langle h \rangle", \langle r.id \rangle, s)$                                                                                                           |
|          | $URI\_track(r, s) \equiv hasURI("http://musicbrainz.org/track/\langle h \rangle", \langle r.id \rangle, s)$                                                                                                             |
| $\Psi_1$ | $mo:MusicArtist(s) \leftarrow artist(r), URI\_artist(r, s)$                                                                                                                                                             |
| $\Psi_2$ | $mo:SoloMusicArtist(s) \leftarrow artist(r), URI\_artist(r, s), (r.type = 1)$                                                                                                                                           |
| $\Psi_3$ | $mo:MusicGroup(s) \leftarrow artist(r), URI\_artist(r, s), (r.type = 2)$                                                                                                                                                |
| $\Psi_4$ | $mo:Record(s) \leftarrow medium(r), URI\_medium(r, s)$                                                                                                                                                                  |
| $\Psi_5$ | $mo:Track(s) \leftarrow track(r), URI\_track(r, s)$                                                                                                                                                                     |
| $\Psi_6$ | $foaf:made(s, g) \leftarrow artist(r), URI\_artist(r, s), track(t), URI\_track(t, g),$<br>$[(artist\_credit\_name\_fk\_artist)^{-1}, artist\_credit\_name\_fk\_artist\_credit, (track\_fk\_artist\_credit)^{-1}](r, t)$ |
| $\Psi_7$ | $mo:track(s, g) \leftarrow medium(r), URI\_medium(r, s), track(t), URI\_track(t, g), [(track\_fk\_medium)^{-1}](r, t)$                                                                                                  |
| $\Psi_8$ | $foaf:name(s, v) \leftarrow artist(r), URI\_artist(r, s), nonNull(r.name), RDFLiteral(r.name, "name", "artist", v)$                                                                                                     |
| $\Psi_9$ | $foaf:gender(s, v) \leftarrow artist(r), URI\_artist(r, s), gender(g), nonNull(g.name), lower(g.name, u)$<br>$RDFLiteral(u, "name", "gender", v), [artist\_fk\_gender](r, g),$                                          |

### 3.2 Examples of Transformation Rules

The experiments use an RDB2RML view defined on top of the *MusicBrainz* database. The view adopts the *Music Ontology* vocabulary and is defined using R2RML statements, which are automatically compiled into transformation rules (see Section ??).

Figure 1(a) shows the *MusicBrainz* schema, Figure 1(b) illustrates a fragment of the *Music Ontology*, and Table 2 lists some of the transformation rules compiled from the R2RML statements.

Table 2 is organized as follows:

*URI definitions.* The first three lines define how the URIs are constructed.

*Class Transformation Rules (CTRs).*  $\Psi_1$  through  $\Psi_5$  are CTRs that define the instances of classes *mo:MusicArtist* through *mo:Track*.

*Object Property Transformation Rules (OTRs).*  $\Psi_6$  and  $\Psi_7$  are OTRs. The body of  $\Psi_6$  has a predicate path

$$[(\text{artist\_credit\_name\_fk\_artist})^{-1}, \\ \text{artist\_credit\_name\_fk\_artist\_credit}, \\ (\text{track\_fk\_artist\_credit})^{-1}](r, t)$$

composed of three foreign keys (defined in Figure 1(a)) that connects tuples  $r$  of relation *artist* with tuples  $t$  of relation *track*. Note that the first and the third foreign keys should be traversed in the reverse direction.

The second OTR,  $\Psi_7$ , has a predicate path

$$[(\text{track\_fk\_medium})^{-1}](r, t)$$

composed of a single foreign key (again defined in Figure 1(a)) that connects tuples  $r$  of relation *medium* with tuples  $t$  of relation *track*. Again, the foreign key should be traversed in the reverse direction.

*Datatype Property Transformation Rules (DTRs).*  $\Psi_8$  is a Local DTR that defines the datatype property *foaf:name* as the name of an artist, whereas  $\Psi_9$  is a Path DTR that defines the datatype property *foaf:gender* as the artistic gender of an artist, using table *gender*.

### 3.3 Hierarchical RDF-State Semantics of RDB2RDF Views

Let  $\mathcal{W} = (V, \mathcal{S}, M)$  be an RDB2RDF view and  $\sigma$  be a database state.

The *state* of  $\mathcal{W}$  in  $\sigma$  is the set of quads  $(s, p, o, g)$  generated by the TRs in  $M$  when applied to  $\sigma$ , where the fourth element  $g$  is the URI of the TR that generated the triple  $(s, p, o)$ . Intuitively, each TR  $\Psi$  induces a named graph containing exactly the triples generated by  $\Psi$ , which provides a simple and operational mechanism for provenance tracking and avoids ambiguity in the presence of duplicate triples generated by different TRs. The state of  $\mathcal{W}$  is defined compositionally in three hierarchical levels.

Let  $\Psi \in M$  be a TR with tuple variables  $r_1, \dots, r_n$  associated with relation schemes  $R_1, \dots, R_n$ . For tuples  $p_1 \in R_1(\sigma), \dots, p_n \in R_n(\sigma)$ , the *application of  $\Psi$  under  $\sigma$*  is the set of RDF triples produced by evaluating  $\Psi$  with bindings  $r_i = p_i$ , for  $i = 1, \dots, n$ , over  $\sigma$ , denoted

$$\Psi[r_1 \leftarrow p_1, \dots, r_n \leftarrow p_n](\sigma)$$

Assume that the pivot relation of  $\Psi$  is  $R^*$ . For a pivot tuple  $t \in R^*(\sigma)$ , the *RDF contribution of  $t$  under  $\Psi$  and  $\sigma$*  is the set:

$$\text{RDF\_State}[\Psi](t, \sigma) = \\ \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow t](\sigma), g = \text{URI}(\Psi)\}$$

Finally, the *RDF state of the view  $\mathcal{W}$  under  $\sigma$*  is the set

$$\text{RDF\_State}(\mathcal{W}, \sigma) = \bigcup_{\Psi \in M} \bigcup_{t \in \text{pivot}(\Psi)(\sigma)} \text{RDF\_State}[\Psi](t, \sigma)$$

That is, the state of an RDB2RDF view is obtained by aggregating the RDF contributions generated by all TRs for all pivot tuples in the database state.

## 4 Post-Update Incremental Maintenance of RDB2RDF Views

### 4.1 The Incremental Maintenance Problem

An *update* on a relation  $R$  is a pair  $u = (D, I)$ , where  $D$  and  $I$  are (possibly empty) sets of tuples, indicating that the tuples in  $D$  must be first deleted from  $R$  and then the tuples in  $I$  must be inserted into  $R$ . The application of  $u$  induces a transition from a *pre-update* database state  $\sigma_0$  to the *post-update* state  $\sigma_1$ .

Let  $u$  be an update and  $\sigma_0$  and  $\sigma_1$  be the pre-update and post-update database states of  $u$ . Let  $W_0 = \text{RDF\_State}(\mathcal{W}, \sigma_0)$  and  $W_1 = \text{RDF\_State}(\mathcal{W}, \sigma_1)$  be the view states before and after the update  $u$ .

A *view changeset* is a pair  $\langle \Delta^-(u), \Delta^+(u) \rangle$ , where  $\Delta^-(u)$  is the set of quads to be removed from  $W_0$ , called the *deletion changeset*, and  $\Delta^+(u)$  is the set of quads to be inserted into  $W_0$ , after deleting the quads in  $\Delta^-(u)$ , called the *insertion changeset*.

The view changeset is *correct* iff

$$W_1 = (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

In other words, the changeset  $\langle \Delta^-(u), \Delta^+(u) \rangle$  is correct if applying it to the pre-update view state  $W_0$  yields the same result as recomputing the view over  $\sigma_1$ .

An approach for computing changesets is to use database triggers. However, in relational systems, transition tables such as *deleted\_R* and *inserted\_R* are only available in *AFTER statement-level triggers*, where the database is already in the post-update state  $\sigma_1$ . As a consequence, the pre-update state  $\sigma_0$  is no longer directly accessible. A key limitation of previous approaches is that they often require reconstructing portions of the pre-update state of the updated relation  $R$  to compute  $\Delta^-(u)$ . This reconstruction can be expensive and impractical, particularly for large relations. Section 4.2 then introduces a strategy for computing  $\Delta^-(u)$  and  $\Delta^+(u)$  that avoids this problem.

### 4.2 A Post-Update Formulation of Changeset Computation

This section presents an algorithm to compute the insertion and deletion changesets  $\Delta^-(u)$  and  $\Delta^+(u)$ . The algorithm basically organizes the definitions in [3] in pseudocode, following a compositional structure, and ensuring that all affected RDF quads are computed using only the post-update state, the tuples recorded in the transition tables, and the pre-update RDB2RDF view state.

Let  $\mathcal{W} = (V, \mathcal{S}, M)$  be an RDB2RDF view and  $\Psi$  be a transformation rule (TR) in  $M$ . Let  $u = (D, I)$  be an update over a relation  $R$ , and  $\sigma_1$  be the database state after applying  $u$ .

The algorithm proceeds by decomposing the computation of the changeset into TR-level contributions. The first step is to identify the TRs that are relevant to updates on  $R$ :

- (1)  $\Psi$  is *pivot-relevant* to updates on  $R$  iff  $\text{pivot}(\Psi) = R$ , that is,  $R$  is the first relation in the body of  $\Psi$ .
- (2)  $\Psi$  is *relation-relevant* to updates on  $R$  iff  $R$  occurs in the body of  $\Psi$  in an occurrence other than the pivot occurrence.
- (3)  $\Psi$  is *relevant* to updates on  $R$  iff  $\Psi$  is pivot-relevant or relation-relevant to updates on  $R$ ;  $\text{Relev}(R)$  denotes the set of transformation rules relevant to updates on  $R$ .

Note that  $\Psi$  may be both pivot-relevant and relation-relevant to  $R$ . If  $\Psi$  is pivot-relevant to updates on  $R$ , the tuples in  $D$  and  $I$  are themselves pivot tuples of  $\Psi$ , and their RDF contributions are directly affected by the update. If  $\Psi$  is relation-relevant to updates on  $R$ , the update affects the evaluation of  $\Psi$  indirectly: tuples in the pivot relation  $R^*$  of  $\Psi$  whose derivations depend on tuples in  $D$  or  $I$  may have their RDF contributions modified.

For example, consider the TR  $\Psi_6$  in Table 2. Then,  $\Psi_6$  is relevant to *artist*, which is the pivot relation of  $\Psi_6$ , to *track*, which explicitly occurs in *path*( $\Psi_6$ ), and to *artist\_credit\_name* and *artist\_credit*, which are used in foreign keys that occur in *path*( $\Psi_6$ ).

Next, for each rule  $\Psi \in \text{Relev}(R)$ , the algorithm computes the contribution of  $\Psi$  to the changesets by deriving the deletion and insertion components, denoted  $\Delta^-[\Psi](u)$  and  $\Delta^+[\Psi](u)$ , respectively, which capture the RDF quads that must be removed from and added to the view, using  $\Psi$ , with respect to  $u$ .

The overall changeset is then obtained by aggregating the contributions of all relevant rules:

$$\Delta^-(u) = \bigcup_{\Psi \in \text{Relev}(R)} \Delta^-[\Psi](u) \quad \Delta^+(u) = \bigcup_{\Psi \in \text{Relev}(R)} \Delta^+[\Psi](u)$$

The key idea is that changesets can be computed by characterizing the effect of an update at the level of each relevant TR contribution.

The deletion and insertion components,  $\Delta^-[\Psi](u)$  and  $\Delta^+[\Psi](u)$ , in turn have two components:

$$\Delta^-[\Psi](u) = \Delta_{\text{pivot}}^-[\Psi](u) \cup \Delta_{\text{rel}}^-[\Psi](u)$$

$$\Delta^+[\Psi](u) = \Delta_{\text{pivot}}^+[\Psi](u) \cup \Delta_{\text{rel}}^+[\Psi](u)$$

where:

- $\Delta_{\text{pivot}}^\pm[\Psi](u)$  denotes the contribution from the pivot-relevant effect of  $\Psi$ .
- $\Delta_{\text{rel}}^\pm[\Psi](u)$  denotes the contribution from the relation-relevant effect of  $\Psi$ .

The rest of this section briefly explains the major steps of Algorithm 1.

### Step 1: Computing the pivot-relevant contribution (Lines 9–18).

If  $\Psi$  is pivot-relevant to  $R$ , then the tuples in  $D$  and  $I$  are themselves pivot tuples of  $\Psi$ .

In this case, the deletion contribution  $\Delta_{\text{pivot}}^-[\Psi](u)$  consists of the quads in the pre-update RDF state  $W_0$  generated by  $\Psi$  from deleted pivot tuples  $d \in D$ . These quads become invalid after the update and must be removed from the view.

Symmetrically, the insertion contribution  $\Delta_{\text{pivot}}^+[\Psi](u)$  consists of the quads generated by applying  $\Psi$  to inserted pivot tuples  $i \in I$ , evaluated over the post-update database state  $\sigma_1$ . These quads become valid after the update and must be inserted into the view.

### Step 2: Computing the relation-relevant contribution (Lines 19–39).

If  $\Psi$  is relation-relevant to  $R$ , the update affects the evaluation of  $\Psi$  indirectly. In this case, tuples in the pivot relation  $R^* = \text{pivot}(\Psi)$  whose derivations depend on tuples in  $D$  or  $I$  may have their RDF contributions modified. The computation proceeds in two steps.

#### Step 2.1: Affected pivot tuples for deletion (Lines 21–33).

Compute  $A_\Psi^-$ , the sets of pivot tuples in  $R^*$  affected by the deleted tuples in  $D$ . Two cases must be distinguished:

- **Single occurrence of  $R$  in *path*( $\Psi$ ) (Lines 21–23):** the affected pivot tuples can be identified directly from the post-update state  $\sigma_1$ , without simulating the pre-update state of  $R$ .
- **Multiple occurrences of  $R$  in *path*( $\Psi$ ) (Lines 24–27):** the computation simulates the pre-update state of  $R$  by binding, in  $\Psi$ , occurrences of tuple variables associated with  $R$  to tuples in the post-update state of  $R$  or in  $D$ , but not in  $I$  (Lines 25 and 26), while binding the occurrences of any other relation  $S$  to tuples in the post-update state of  $S$ .

Intuitively, the affected sets contain the pivot tuples whose RDF derivations depend on tuples modified by the update.

For each affected pivot tuple  $p^* \in A_\Psi^-$ , define:

- $S_1[\Psi](p^*)$ : quads created by  $\Psi$  from  $p^*$  in the pre-update RDF state  $W_0$ .
- $S_2[\Psi](p^*)$ : quads created by  $\Psi$  from  $p^*$  in the post-update database state  $\sigma_1$ .

The deletion contribution associated with  $p^*$  is then:

$$\Delta_{\text{rel}}^-[\Psi](p^*) = S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)$$

Intuitively, the set difference removes from the view exactly the quads that were valid before the update but are no longer generated after the update.

#### Step 2.2: Affected pivot tuples for insertion (Lines 34–38).

Symmetrically, for each affected pivot tuple  $p^* \in A_\Psi^+$ , the insertion contribution is defined as the set of quads generated by  $\Psi$  from  $p^*$  in the post-update state  $\sigma_1$ :

$$\Delta_{\text{rel}}^+[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = \text{URI}(\Psi)\}$$

### Step 3: Overall deletion and insertion contributions (Lines 40–43).

The overall deletion and insertion contributions of  $\Psi$  are then obtained from the combined pivot and relation-relevant contributions:

$$\Delta^-[\Psi](u) = \Delta_{\text{pivot}}^-[\Psi](u) \cup \Delta_{\text{rel}}^-[\Psi](u)$$

$$\Delta^+[\Psi](u) = \Delta_{\text{pivot}}^+[\Psi](u) \cup \Delta_{\text{rel}}^+[\Psi](p^*)$$

This concludes the brief remarks on Algorithm 1.

Theorem 1 states that Algorithm 1 outputs correct insertion and deletion changesets  $\Delta^-(u)$  and  $\Delta^+(u)$  (see [3] for a proof).

Let  $\mathcal{W} = (V, \mathcal{S}, M)$  be an RDB2RDF view. Let  $u = (D, I)$  be an update on a relation  $R$ ,  $\sigma_0$  and  $\sigma_1$  be the database states before and after applying  $u$ , and  $W_0 = \text{RDF\_State}(\mathcal{W}, \sigma_0)$  and  $W_1 = \text{RDF\_State}(\mathcal{W}, \sigma_1)$  be the view states before and after applying  $u$ . Let  $\Delta^-(u)$  and  $\Delta^+(u)$  be the deletion and insertion components computed as in Algorithm 1.

### Theorem 1 (Correctness of the Incremental Changesets).

$$W_1 = (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

**Algorithm 1:** COMPUTE\_DELTA\_R( $W, u, R, \sigma_1, W_0$ )**Input:** RDB2RDF view  $W = (V, S, M)$ ; update  $u = (D, I)$  over relation  $R$ ; post-update database state  $\sigma_1$ ; pre-update RDF state  $W_0$ .**Output:** Changeset  $\langle \Delta^-(u), \Delta^+(u) \rangle$ .

```

1.  $\Delta^-(u) \leftarrow \emptyset$ ;
2.  $\Delta^+(u) \leftarrow \emptyset$ ;
3.  $Relev(R) \leftarrow \{\Psi \in M \mid \Psi \text{ is pivot-relevant to } R \vee \Psi \text{ is relation-relevant to } R\}$ ;
4. foreach  $\Psi \in Relev(R)$  do
5.    $\Delta_{pivot}^-(\Psi)(u) \leftarrow \emptyset$ ;
6.    $\Delta_{rel}^-(\Psi)(u) \leftarrow \emptyset$ ;
7.    $\Delta_{pivot}^+(\Psi)(u) \leftarrow \emptyset$ ;
8.    $\Delta_{rel}^+(\Psi)(u) \leftarrow \emptyset$ ;
   // 1. Pivot-relevant contribution
9.   if  $\Psi$  is pivot-relevant to updates on  $R$  then
10.    foreach  $d \in D$  do
11.       $\Delta_{pivot}^-(\Psi)(d) \leftarrow \{(s, p, o, g) \in W_0 \mid s = URI(d) \wedge g = URI(\Psi)\}$ ;
12.       $\Delta_{pivot}^-(\Psi)(u) \leftarrow \Delta_{pivot}^-(\Psi)(u) \cup \Delta_{pivot}^-(\Psi)(d)$ ;
13.    end
14.    foreach  $i \in I$  do
15.       $\Delta_{pivot}^+(\Psi)(i) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow i](\sigma_1) \wedge g = URI(\Psi)\}$ ;
16.       $\Delta_{pivot}^+(\Psi)(u) \leftarrow \Delta_{pivot}^+(\Psi)(u) \cup \Delta_{pivot}^+(\Psi)(i)$ ;
17.    end
18.  end
   // 2. Relation-relevant contribution
19.  if  $\Psi$  is relation-relevant to updates on  $R$  then
20.     $R^* \leftarrow pivot(\Psi)$ ;
    // 2.1 Affected pivot tuples for deletion
21.    if  $R$  occurs only once in  $path(\Psi)$  then
22.       $A_{\Psi}^- \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_R \leftarrow d](\sigma_1)\}$ ;
23.    end
24.    else
25.       $\sigma_R^0 \leftarrow (R(\sigma_1) \cup D) \setminus I$ ;
26.       $A_{\Psi}^- \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists r_k \in Var_R(\Psi), \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1[R \leftarrow \sigma_R^0])\}$ ;
27.    end
28.    foreach  $p^* \in A_{\Psi}^-$  do
29.       $S_1[\Psi](p^*) \leftarrow \{(s, p, o, g) \in W_0 \mid s = URI(p^*) \wedge g = URI(\Psi)\}$ ;
30.       $S_2[\Psi](p^*) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$ ;
31.       $\Delta_{rel}^-(\Psi)(p^*) \leftarrow S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)$ ;
32.       $\Delta_{rel}^-(\Psi)(u) \leftarrow \Delta_{rel}^-(\Psi)(u) \cup \Delta_{rel}^-(\Psi)(p^*)$ ;
33.    end
    // 2.2 Affected pivot tuples for insertion
34.     $A_{\Psi}^+ \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \exists r_k \in Var_R(\Psi), \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow i](\sigma_1)\}$ ;
35.    foreach  $p^* \in A_{\Psi}^+$  do
36.       $\Delta_{rel}^+(\Psi)(p^*) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$ ;
37.       $\Delta_{rel}^+(\Psi)(u) \leftarrow \Delta_{rel}^+(\Psi)(u) \cup \Delta_{rel}^+(\Psi)(p^*)$ ;
38.    end
39.  end
   // 3. Overall deletion and insertion contributions
40.   $\Delta^-(\Psi)(u) \leftarrow \Delta_{pivot}^-(\Psi)(u) \cup \Delta_{rel}^-(\Psi)(u)$ ;
41.   $\Delta^+(\Psi)(u) \leftarrow \Delta_{pivot}^+(\Psi)(u) \cup \Delta_{rel}^+(\Psi)(u)$ ;
42.   $\Delta^-(u) \leftarrow \Delta^-(u) \cup \Delta^-(\Psi)(u)$ ;
43.   $\Delta^+(u) \leftarrow \Delta^+(u) \cup \Delta^+(\Psi)(u)$ ;
44. end
45. return  $\langle \Delta^-(u), \Delta^+(u) \rangle$ ;

```



Finally, Theorem 1 has a simple corollary. Let  $\sigma_D$  be the database state after applying the deletions in  $D$ , but before applying the insertions in  $I$ . Let  $W_D = RDF\_State(\mathcal{W}, \sigma_D)$  be the view state corresponding to  $\sigma_D$ .

#### Corollary 1 (Minimality of the Incremental Changesets).

- (a)  $\Delta^-(u) = (W_0 \setminus W_D)$
- (b)  $(\Delta^+(u) \setminus W_D) = (W_1 \setminus W_D)$
- (c) If  $(\Delta^+(u) \cap W_D) = \emptyset$ , then  $\Delta^+(u) = (W_1 \setminus W_D)$

Corollary 1 suggests that the changesets are minimal in the following sense. Part (a) says that  $\Delta^-(u)$  is exactly the set of quads that are removed from the initial view state  $W_0$  as a result of the deletions in the update  $u$  (but before the insertions). Likewise, Part (b) states that  $\Delta^+(u)$  is the set of quads that are inserted into the intermediate view state  $W_D$ , created after the deletions, as a result of the insertions in the update  $u$ . But we have to ignore quads that are already in  $W_D$ , since we do not rule out stale insertions that lead to duplicates. Part (c) is a simple consequence of Part (b) and says that, if  $\Delta^+(u)$  has no duplicates w.r.t.  $W_D$ , then  $\Delta^+(u)$  is exactly the set of quads that must be inserted into  $W_D$  to create  $W_1$ .

### 4.3 An Asynchronous Architecture

Algorithm 1 defines how to compute the changesets  $\Delta^-(u)$  and  $\Delta^+(u)$ , but it must be re-engineered to separate accessing the post-update database state  $\sigma_1$  from accessing the pre-update view state  $W_0$ , since the database and the view are typically in distinct environments. This section then explores an architecture, shown in Figure 2, that addresses this problem.

The architecture combines AFTER triggers with an asynchronous maintenance worker. Instead of directly computing and applying the changeset inside a relational transaction, a trigger captures a snapshot of the update event on a relation  $R$  (the transition tables  $D$  and  $I$ ), and computes the affected tuples for deletion  $A^-[\Psi](u)$ , the post-update contributions  $S_2[\Psi](u)$  to  $\Delta^-(u)$ , and the insertion changeset  $\Delta^+(u)$  (which requires computing the affected tuples for insertion  $A^+[\Psi](u)$ ). This information is then stored in a maintenance queue. The AFTER trigger for a relation  $R$  calls the function `COMPUTE_CHANGESET_R`, shown in Algorithm 2, to compute  $A^-[\Psi](u)$ ,  $S_2[\Psi](u)$ , and  $\Delta^+[\Psi](u)$  and update the maintenance queue.

An external worker processes queued events sequentially, preserving the update commit order. For each event, the worker first queries the triplestore to retrieve from the RDB2RDF view state  $W_0$  the corresponding deletion contribution of deleted tuples,  $\Delta_{pivot}^-[\Psi](u)$ , and the pre-update RDF contributions,  $S_1[\Psi](u)$ . Then, it computes the deletion contribution using the set difference  $(S_1[\Psi](u) \setminus S_2[\Psi](u))$ , applies  $\Delta^-(u)$  to the view, and then applies  $\Delta^+(u)$ .

This architecture offers several advantages. First, it avoids executing SPARQL queries inside relational transactions, preventing triplestore latency or temporary unavailability from blocking database updates. Second, it preserves correctness by storing the relational context of the update at commit time, ensuring that subsequent database updates do not affect the computation of the changeset. Third, the queue-based design improves scalability, fault

tolerance, and recoverability, since maintenance events can be re-tried or processed asynchronously without impacting transactional workloads.

## 5 Implementation

### 5.1 Motivation

The formal framework presented in the previous sections defines a complete transformation from Entity-Preserving mappings into executable triggers for incremental RDB2RDF view maintenance. A natural question arising from this theoretical development is whether such a compilation process can be realized automatically in practice. To answer this question, this section introduces a prototype compiler that, starting with R2RML mappings that define an entity-preserving RDB2RDF view, instantiates the proposed framework using Large Language Models (LLMs).

The compiler follows a two-phase compilation process. Each compilation phase is implemented by a specialized LLM-based compiler module responsible for generating a well-defined intermediate artifact. Since LLMs may occasionally produce incomplete or inconsistent outputs, each generated artifact is independently verified before being accepted as the input of the subsequent compilation phase. This verification process ensures that every compilation step conforms to the formal rules defined by the proposed framework.

### 5.2 Phase 1 – Transformation Rules Compilation

The first compilation phase transforms an Entity-Preserving R2RML mapping into a certified semantic specification expressed as Transformation Rules (TRs). This phase is implemented by the Semantic Compilation Agent, guided by a prompt (available at the supplemental material), whose inputs consist of the relational database schema, and the R2RML mapping.

The Semantic Compilation Agent performs the following tasks:

- (1) Verify the Entity-Preserving property.
- (2) Identify the pivot relation.
- (3) Derive the URI constructor.
- (4) Define the corresponding hasURI predicate.
- (5) Validate the generated semantic specification.
- (6) Generate the Transformation Rules.

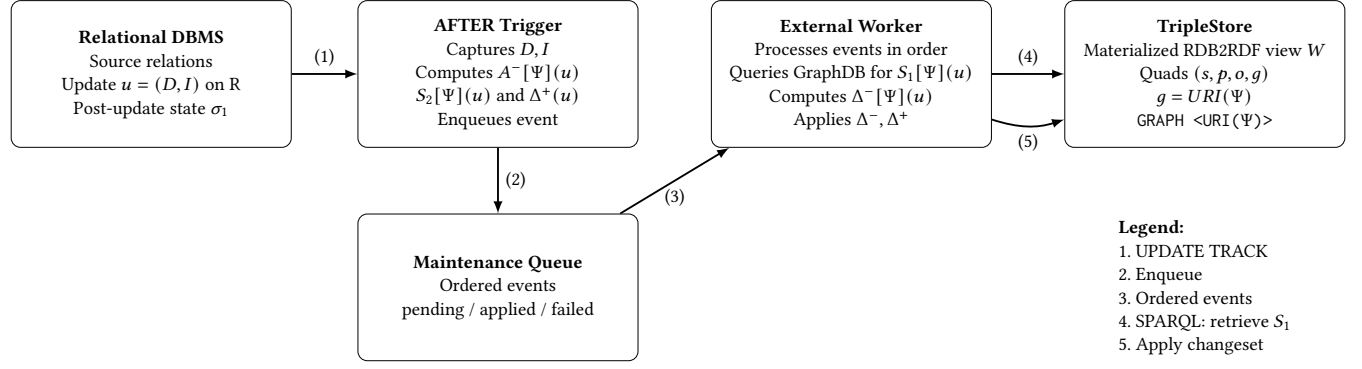
The output of this phase is a structured JSON specification representing a certified intermediate artifact. Mappings that fail validation are reported together with the detected inconsistencies and marked as “Manual Analysis Required”. No Transformation Rules are generated for invalid mappings.

This intermediate representation completely separates semantic reasoning from code generation. Consequently, the second compilation phase no longer needs to interpret the R2RML mapping, relying exclusively on the certified semantic specification.

### 5.3 Phase 2 – Trigger Compilation

The second compilation phase transforms the Transformation Specification into AFTER triggers, as discussed in Section 4.3.

This phase is implemented by the Trigger Compilation Agent, guided by a prompt (available at the supplemental material), whose



**Figure 2: Asynchronous architecture for incremental maintenance of a materialized RDB2RDF view.** AFTER triggers capture the transition tables  $D$  and  $I$ , and compute  $A^-[\Psi](u)$ ,  $S_2[\Psi](u)$ , and  $\Delta^+[\Psi](u)$  (which requires computing  $A^+[\Psi](u)$ ). An external worker processes queued events in commit order, retrieves  $\Delta_{pivot}^-[\Psi](u)$  and  $S_1[\Psi](u)$  from the triplestore, computes  $\Delta^-[\Psi](u)$ , and updates the RDF materialization.

---

**Algorithm 2:** Function COMPUTE\_CHANGESSET\_R for the Asynchronous Architecture

---

**Input:** RDB2RDF view  $W = (V, S, M)$ ; update  $u = (D, I)$  over relation  $R$ .

**Output:** Insertions on tables *rdf\_maintenance\_queue* and *rdf\_rule\_contribution*.

// Event-level queue initialization:

1. Insert one row into *rdf\_maintenance\_queue* for the statement-level update  $u = (D, I)$ ;
  - // Rule-level computation and publication:
  2. **foreach**  $\Psi \in \text{Relev}(R)$  **do**
    - // Execute the following two steps sequentially:
    - // STEP 1 – Incremental Maintenance Computation for  $\Psi$
    - 3.  $\Delta_{pivot}^+[\Psi](u) \leftarrow \emptyset$ ;
    - 4.  $\Delta_{rel}^+[\Psi](u) \leftarrow \emptyset$ ;
    - 5. **if**  $\Psi$  is pivot-relevant to updates on  $R$  **then**
      - 6. Compute  $\Delta_{pivot}^+[\Psi](u)$ ;
    - 7. **end**
    - 8. **if**  $\Psi$  is relation-relevant to updates on  $R$  **then**
      - 9. Compute  $A^-[\Psi](u)$ ,  $A^+[\Psi](u)$ ,  $S_2[\Psi](u)$ , and  $\Delta_{rel}^+[\Psi](u)$ ;
    - 10. **end**
    - 11.  $\Delta^+[\Psi](u) \leftarrow \Delta_{pivot}^+[\Psi](u) \cup \Delta_{rel}^+[\Psi](u)$ ;
    - // STEP 2 – Rule Contribution Publication
    - 12. Insert one row into *rdf\_rule\_contribution* for  $\Psi$  with  $A^-[\Psi](u)$ ,  $S_2[\Psi](u)$ , and  $\Delta^+[\Psi](u)$
    - 13. linked to the *event\_id* created in *rdf\_maintenance\_queue*;
  14. **end**
- 

inputs consist of the semantic specification generated during Phase 1, and the incremental maintenance framework of Section 4.3.

Each trigger captures a snapshot of the update event on a relation  $R$  (the transition tables  $D$  and  $I$ ), and computes the affected tuples for deletion  $A^-[\Psi](u)$ , the post-update contributions  $S_2[\Psi](u)$  to  $\Delta^-(u)$ , and the insertion changeset  $\Delta^+(u)$  (which requires computing the affected tuples for insertion  $A^+[\Psi](u)$ ). Table 2 contains URLs with examples of the LLM-generated trigger code.

Finally, each generated artifact is subsequently validated.

## 6 Experiments

### 6.1 Experimental Setup

**Data and LLMs.** The experiments were based on:

- *Relational Database*: a local copy of the *MusicBrainz* database, stored in PostgreSQL.
- *LinkedBrainz RDB2RDF View*: an RDB2RDF view of the *MusicBrainz* database, materialized in GraphDB.
- *Selected relations*: the relations **TRACK**, **ARTIST**, and **MEDIUM** were chosen because they occur in transformation rules with complex paths.

- **Selected updates:** a list with one insertion, one deletion, and one update per selected relation, along with their expected changesets.
- **Selected LLMs:** gpt-5.3-chat, DeepSeek V3.2, and Qwen3.6-Plus.

MusicBrainz<sup>2</sup> is a large and actively maintained open music knowledge base operated by the MetaBrainz Foundation. The complete relational schema<sup>3</sup> comprises 374 tables, with data about artists, release groups, etc. The database is implemented in PostgreSQL and continuously publishes incremental database updates through its replication mechanism, providing a realistic scenario for high-frequency relational changes. This makes the case study particularly suitable for evaluating incremental maintenance techniques for materialized RDB2RDF views.

The LinkedBrainz RDB2RDF View uses the Music Ontology vocabulary<sup>4</sup> and is specified in R2RML through 57 TriplesMaps<sup>5</sup>. Although MusicBrainz distributes incremental relational updates through replication packets, maintaining a materialized view over MusicBrainz still requires computing changesets that reflect the impact of relational updates on the RDB2RDF materialized view.

**Validation of Triggers Generated by LLMs<sup>6</sup>.** The experimental evaluation was conducted in two stages:

#### Stage 1 – Validation of the Generated Trigger Code

In Stage 1, each selected LLM was prompted to generate the complete set of PostgreSQL AFTER triggers described in Section 5.2. For each combination of LLM and database relation, the generated trigger code was manually evaluated according to the criteria presented in Table 3.

#### Stage 2 – Validation of Incremental Maintenance Results

In Stage 2, the trigger code generated in Stage 1 was executed using different update scenarios. The goal was to evaluate the execution correctness of the generated triggers, i.e., if they compute the expected incremental maintenance results for each update.

Execution correctness was assessed according to two complementary criteria.

**Rule Contribution Correctness.** This criterion evaluates whether the generated trigger computes the expected maintenance information for each relevant transformation rule and update scenario. For each update  $u$  and each relevant transformation rule  $\Psi \in \text{Relev}(R)$ , the generated rule contributions

$$A^-[\Psi], A^+[\Psi], S2[\Psi], \Delta_{\text{pivot}}^+[\Psi], \Delta_{\text{rel}}^+[\Psi], \Delta^+[\Psi]$$

were manually compared against the expected results derived from the formal incremental maintenance semantics of Section 4.

**Changeset Correctness.** This criterion evaluates whether the generated trigger computes the correct RDF insertion and deletion changesets. To ensure an unbiased assessment, correctness was verified against an independently computed oracle obtained by recomputing the full RDF view after each update.

Let  $W_0$  denote the RDF view before the update and  $W_1^{\text{oracle}} = M(\sigma_1)$  the RDF view obtained by applying the complete RDB2RDF mapping to the updated database state  $\sigma_1$ . The generated trigger is considered correct if the RDF view reconstructed from its computed changesets satisfies:

$$W_1^{\text{oracle}} = (W_0 - \Delta^-(u)) \cup \Delta^+(u)$$

This criterion provides an end-to-end validation of the generated maintenance code by verifying that it produces exactly the same RDF view as full recomputation.

Note that, to avoid a combinatorial explosion, the (manual) evaluation of the results was constrained to use the selected LLMs and to cover only the selected relations and updates. **Stage 1 called each LLM through its free-tier chat interface: gpt-5.3-chat, DeepSeek in “instant” mode, and Qwen3.6-Plus in “auto” mode, referred to as GPT, DeepSeek, and Qwen in the tables to save column space.**

**Table 3: Evaluation Criteria.**

| Stage | Dimension                          | Description                                                                                                                   |
|-------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1     | TR Identification                  | Does the model correctly identify the relevant transformation rules?                                                          |
|       | Computation of Rules Contributions | Does the model correctly generate the code for the rules contributions?                                                       |
|       | Code Minimality                    | Is the generated code concise and free of unnecessary operations?                                                             |
|       | Hallucinations                     | Does the model introduce unsupported assumptions or irrelevant joins?                                                         |
| 2     | Rule Contribution Correctness      | Does the generated trigger compute the expected rule contributions for each relevant transformation rule and update scenario? |
|       | Changeset Correctness              | Does the generated trigger compute the expected RDF changesets for each update scenario?                                      |

**Table 4: Session URLs with the AFTER Triggers generated.**

| Model    | Rel.   | Generated AFTER Trigger – Session URL                                                                                                                           |
|----------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GPT      | TRACK  | <a href="https://chatgpt.com/s/t_6a24b2089e448191869798d0f1969911">https://chatgpt.com/s/t_6a24b2089e448191869798d0f1969911</a>                                 |
|          | ARTIST | <a href="https://chatgpt.com/s/t_6a24c7055b48819198c777b012ed77b2">https://chatgpt.com/s/t_6a24c7055b48819198c777b012ed77b2</a>                                 |
|          | MEDIUM |                                                                                                                                                                 |
| DeepSeek | TRACK  | <a href="https://chat.deepseek.com/share/jy62rtm11ie7sl78yt">https://chat.deepseek.com/share/jy62rtm11ie7sl78yt</a>                                             |
|          | ARTIST |                                                                                                                                                                 |
|          | MEDIUM |                                                                                                                                                                 |
| Qwen     | TRACK  | <a href="https://chat.qwen.ai/s/t_104106fe-1842-47ac-97be-9c6eb1e83c2a?fev=0.2.63">https://chat.qwen.ai/s/t_104106fe-1842-47ac-97be-9c6eb1e83c2a?fev=0.2.63</a> |
|          | ARTIST | <a href="https://chat.qwen.ai/s/t_5b0ecb2b-1f7c-429f-81c1-e3850a5c767d?fev=0.2.63">https://chat.qwen.ai/s/t_5b0ecb2b-1f7c-429f-81c1-e3850a5c767d?fev=0.2.63</a> |
|          | MEDIUM |                                                                                                                                                                 |

## 6.2 Results

Table 4 contains the session URLs with the AFTER Triggers generated, and Tables 5 and 6 the manual evaluation for each selected relation. The results suggest that: gpt-5.3-chat generated correct and optimized code in all cases; Qwen3.6-Plus generated correct and concise code in some cases; Deepseek did not fully capture the concepts and semantics of the proposed incremental maintenance framework.

## 7 Conclusions

The first contribution of this paper was a formal definition of the deletion and insertion components of RDB2RDF changesets, designed with LLM-compilation in mind. For a transformation rule without multiple occurrences of the same relation, the definitions

<sup>2</sup>Available at <https://musicbrainz.org>

<sup>3</sup>Available as file MusicBrainzSchema.pdf in the supplemental material.

<sup>4</sup>Available as file MusicBrainzOntoUltimate.owl in the supplemental material.

<sup>5</sup>Available at <https://docs.openlinksw.com/virtuoso/rdfviewsbusintmbr/>

<sup>6</sup>The code adopted in the experiments is available in the supplemental material.

**Table 5: Evaluation for relation TRACK.**

| Stage | Evaluation Criteria                | Model    | Result                                      |
|-------|------------------------------------|----------|---------------------------------------------|
| 1     | Transformation rule Identification | GPT      | identified all relevant rules               |
|       |                                    | DeepSeek | identified all relevant rules               |
|       |                                    | Qwen     | identified all relevant rules               |
| 1     | Computation of Rules Contributions | GPT      | correctly produced all relational joins     |
|       |                                    | DeepSeek | incorrectly produced some relational joins  |
|       |                                    | Qwen     | distinguished two formal cases              |
| 1     | Code Minimality                    | GPT      | produced minimal code                       |
|       |                                    | DeepSeek | introduced redundant intermediate queries   |
|       |                                    | Qwen     | relatively clean and concise                |
| 1     | Hallucinations                     | GPT      | did not hallucinate                         |
|       |                                    | DeepSeek | generated unsupported logic                 |
|       |                                    | Qwen     | limited, but one unsupported simplification |
| 2     | Rule Contribution Correctness      | GPT      |                                             |
|       |                                    | DeepSeek |                                             |
|       |                                    | Qwen     |                                             |
| 2     | Changeset Correctness              | GPT      |                                             |
|       |                                    | DeepSeek |                                             |
|       |                                    | Qwen     |                                             |

**Table 6: Evaluation for relation ARTIST.**

| Stage | Evaluation Criteria                | Model    | Result                                      |
|-------|------------------------------------|----------|---------------------------------------------|
| 1     | Transformation rule Identification | GPT      | identified all relevant rules               |
|       |                                    | DeepSeek | identified all relevant rules               |
|       |                                    | Qwen     | identified all relevant rules               |
| 1     | Computation of Rules Contributions | GPT      | correctly produced all relational joins     |
|       |                                    | DeepSeek | incorrectly produced some relational joins  |
|       |                                    | Qwen     | distinguished two formal cases              |
| 1     | Code Minimality                    | GPT      | produced minimal code                       |
|       |                                    | DeepSeek | introduced redundant intermediate queries   |
|       |                                    | Qwen     | relatively clean and concise                |
| 1     | Hallucinations                     | GPT      | did not hallucinate                         |
|       |                                    | DeepSeek | generated unsupported logic                 |
|       |                                    | Qwen     | limited, but one unsupported simplification |
| 2     | Rule Contribution Correctness      | GPT      |                                             |
|       |                                    | DeepSeek |                                             |
|       |                                    | Qwen     |                                             |
| 2     | Changeset Correctness              | GPT      |                                             |
|       |                                    | DeepSeek |                                             |
|       |                                    | Qwen     |                                             |

**Table 7: Evaluation for relation MEDIUM.**

| Stage | Evaluation Criteria                | Model    | Result                                      |
|-------|------------------------------------|----------|---------------------------------------------|
| 1     | Transformation rule Identification | GPT      | identified all relevant rules               |
|       |                                    | DeepSeek | identified all relevant rules               |
|       |                                    | Qwen     | identified all relevant rules               |
| 1     | Computation of $\Delta^-(u)$       | GPT      | correctly produced all relational joins     |
|       |                                    | DeepSeek | incorrectly produced some relational joins  |
|       |                                    | Qwen     | distinguished two formal cases              |
| 1     | Computation of $\Delta^+(u)$       | GPT      | correctly computed <i>RDF STATE</i>         |
|       |                                    | DeepSeek | generated only partial RDF triples          |
|       |                                    | Qwen     | generated only partial RDF triples          |
| 1     | Code minimality                    | GPT      | produced minimal code                       |
|       |                                    | DeepSeek | introduced redundant intermediate queries   |
|       |                                    | Qwen     | relatively clean and concise                |
| 1     | Hallucinations                     | GPT      | did not hallucinate                         |
|       |                                    | DeepSeek | generated unsupported logic                 |
|       |                                    | Qwen     | limited, but one unsupported simplification |
| 2     | Execution correctness              | GPT      |                                             |
|       |                                    | DeepSeek |                                             |
|       |                                    | Qwen     |                                             |

of both the deletion and insertion components use only the post-update state, transition tables, and the pre-update view state. For a transformation rule with multiple occurrences of the same relation  $R$ , the definitions capture the impact of deletions and insertions on  $R$  directly in the evaluation of the transformation rule, still without using the pre-update database state.

The second contribution was to describe the implementation of an incremental view maintenance tool for RBD2RDF views specified by R2RML statements. The tool is based on a generic architecture that uses AFTER triggers to capture database updates and a specialized worker to maintain the view. The implementation addressed the key question of compiling AFTER triggers from R2RML statements, using transformation rules as an intermediate step.

The third contribution was to carry out an experiment with a real-world database and an RDB2RDF view, specified by R2RML statements, illustrating that an LLM, guided by appropriate prompts and accessing formal definitions, can generate code that computes and publishes the changesets.

As future work, the pre-processing steps described in Section ?? will be re-engineered as a multi-agent system with human-in-the-loop to facilitate compiling R2RML statements into AFTER triggers and installing them in the source relational database.

**Supplemental Material Statement.** The full article, with the formal definitions, results, and proofs, the code, prompts, database, and RDB2RDF view adopted in the experiments are available at <https://anonymous.4open.science/r/llm-assisted-compilation-F7B8>.

**Declaration of use of Generative AI.** In Section 4.2, ChatGPT was used to help check the definitions. In Section 5, ChatGPT was used to help define the architecture. In Section 6, three LLMs were utilized to generate code.

## References

- [1] Mahmoud Abo Khamis, Eden Chmielewski, Andrei Draghici, Ahmet Kara, and Dan Olteanu. 2026. Maintaining Queries under Updates Using Heavy-Light Partitioning of the Input Relations. *Proc. ACM Manag. Data* 4, 2, Article 109 (2026). doi:10.1145/3801905
- [2] M. Akhtar Ali, Alvaro A. A. Fernandes, and Norman W. Paton. 2003. MOVIE: An Incremental Maintenance System for Materialized Object Views. *Data Knowl. Eng.* 47, 2 (Nov. 2003), 131–166.
- [3] Anonymized0. [n. d.]. Anonymized Full Paper. [https://anonymous.4open.science/r/llm-assisted-compilation-F7B8/IVM\\_abril\\_21\\_Current\\_version.pdf](https://anonymous.4open.science/r/llm-assisted-compilation-F7B8/IVM_abril_21_Current_version.pdf)
- [4] Anonymized1. [n. d.]. Anonymized reference 1.
- [5] Anonymized2. [n. d.]. Anonymized reference 2.
- [6] Anonymized3. [n. d.]. Anonymized reference 3.
- [7] Ilaria Battiston, Kriti Kathuria, and Peter Boncz. 2024. OpenVM: a SQL-to-SQL Compiler for Incremental Computations. In *Companion of the 2024 International Conference on Management of Data*. ACM, 516–519. doi:10.1145/3626246.3654743
- [8] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proc. VLDB Endow.* 16, 7 (March 2023), 1601–1614. doi:10.14778/3587136.3587137
- [9] Stefano Ceri and Jennifer Widom. 1991. Deriving Production Rules for Incremental View Maintenance. In *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB '91)*. San Francisco, CA, USA, 577–589.
- [10] Soonbo Han and Zachary G. Ives. 2025. Implementing Views for Property Graphs. *SIGMOD Rec.* 54, 1 (2025), 59–68. doi:10.1145/3733620.3733633
- [11] Xiao Hu and Qichen Wang. 2025. Towards Update-Dependent Analysis of Query Maintenance. *Proc. ACM Manag. Data* 3, 2, Article 117 (2025), 25 pages. doi:10.1145/3725254
- [12] Xueyun Jin and Husheng Liao. 2010. An Algorithm for Incremental Maintenance of Materialized XPath View. In *Proc. 11th International Conf. on Web-age Information Management (WAIM'10)*. Springer-Verlag, Berlin, Heidelberg, 513–524.
- [13] Pengyuan Li, Boris Glavic, Dieter Gawlick, Vasudha Krishnaswamy, Zhen Hua Liu, Danica Porobic, and Xing Niu. 2026. In-memory Incremental Maintenance of Provenance Sketches. In *Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24–27, 2026*. OpenProceedings.org, 56–68. doi:10.48786/EDBT.2026.05
- [14] William Murphy, Nikolaus Holzer, Feitong Qiao, Leyi Cui, Raven Rothkopf, Nathan Koenig, and Mark Santolucito. 2024. Combining LLM Code Generation with Formal Specifications and Reactive Program Synthesis. arXiv:2410.19736 [cs.SE] <https://arxiv.org/abs/2410.19736>
- [15] Dan Olteanu. 2024. Recent Increments in Incremental View Maintenance. arXiv:2404.17679 [cs.DB] <https://arxiv.org/abs/2404.17679>
- [16] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. 2025. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM*



- [17] Aditi Singh, Akash Shetty, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. 2025. A Survey of Large Language Model-Based Generative AI for Text-to-SQL: Benchmarks, Applications, Use Cases, and Challenges. In *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*. 00015–00021. doi:10.1109/CCWC62904.2025.10903689
- [18] Vikash Singh, Darion Cassel, Nathaniel Weir, Nick Feng, and Sam Bayless. 2026. VERGE: Formal Refinement and Guidance Engine for Verifiable LLM Reasoning. arXiv:2601.20055 [cs.CL] <https://arxiv.org/abs/2601.20055>
- [19] Dore Staquet, Bart Buelens, and Jan Van den Bussche. 2026. Incremental View Maintenance for SPARQL Queries: Adapting the Counting Algorithm. *ACM Trans. Web* (Feb. 2026). doi:10.1145/3796549
- [20] Christoforos Svingos, Andre Hernich, Hinnerk Gildhoff, Yannis Papakonstantinou, and Yannis Ioannidis. 2023. Foreign Keys Open the Door for Faster Incremental View Maintenance. *Proc. ACM Manag. Data* 1, 1 (2023), 25 pages. doi:10.1145/3588720
- [21] Maarten Vandenbrande, Ruben Taelman, Pieter Bonte, and Femke Ongenaes. 2025. Incredunica: Web-Based Incremental View Maintenance for SPARQL. In *The Semantic Web*, Edward Curry, Maribel Acosta, Maria Poveda-Villalón, Marieke van Erp, Adegboyega Ojo, Katja Hose, Cogan Shimizu, and Pasquale Lisena (Eds.). Springer Nature Switzerland, Cham, 297–315.
- [22] Kai Wang, Bingcheng Mao, Shuai Jia, Yujie Ding, Dongming Han, Tianyi Ma, and Bin Cao. 2026. SGCR: A Specification-Grounded Framework for Trustworthy LLM Code Review. arXiv:2512.17540 [cs.SE] <https://arxiv.org/abs/2512.17540>
- [23] Ritwik Yadav et al. 2026. Enzyme: Incremental View Maintenance for Data Engineering. arXiv:2603.27775 [cs.DB] <https://arxiv.org/abs/2603.27775>
- [24] Weijie Zhao, Florin Rusu, Bin Dong, Kesheng Wu, and Peter Nugent. 2017. Incremental View Maintenance over Array Data. In *Proc. 2017 ACM International Conference on Management of Data (SIGMOD '17)*. ACM, New York, NY, USA, 139–154.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009